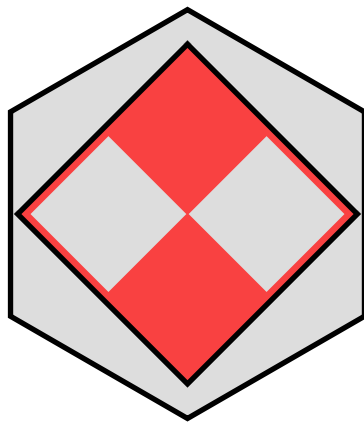




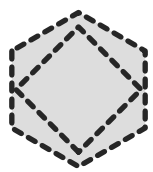
Gridr

Tactics / Strategy - Framework

Gridr is a framework built to help you quickly prototype and produce Turn Based Strategy Games.

To get started you can open up one of the examples to see how a project might be structured and created in a scene. This document will provide an overview, for a more detailed documentation and updates please look at the website.

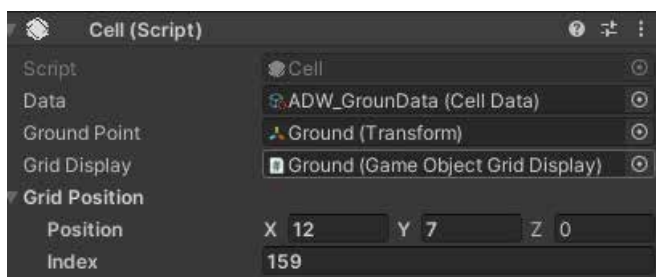
<https://www.generalgames.org>



Cell Component

The Cell component is the basic building block of the game grid. It inherits from MonoBehaviour and can be attached to any GameObject. For the Cell to work as a unit within the grid it is required you attach a CellData asset to the Cell. The CellData is a shared data class that stores information about the cells cost of traversal, whether it is connected in the graph, name and neighbours.

A typical Cell



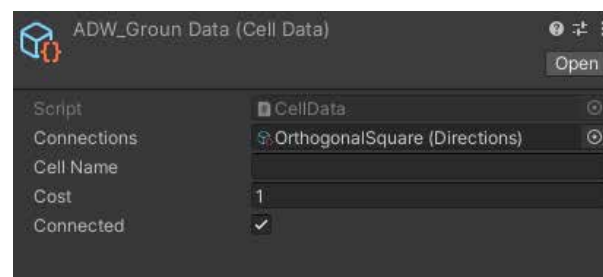
Data: A CellData asset containing shared data

Ground Point: A transform located at the the ground of the cell.

Grid Display: Component responsible for highlighting the cell

GridPosition: A struct that stores the cells position and index within the grid

A typical CellData asset



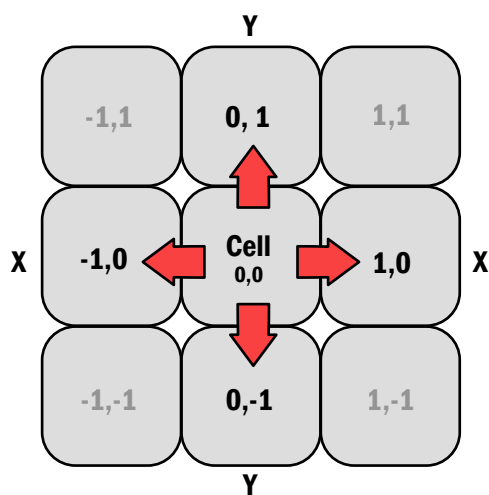
Connections: An array of Direction Assets that determines what cells should be connected to this cell in the graph and the cost multiplier. For example, the Orthogonal data asset contains: (-1,0,0), (0,-1,0), (1,0,0), (0,1,0)

Cell Name: Name of the cell as a string

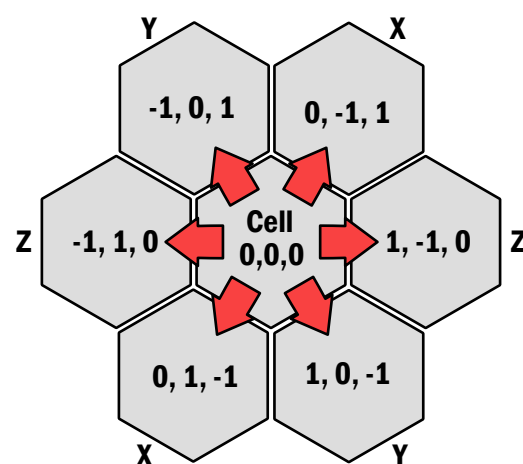
Cost: Edge weight to this node within the graph, the cost of moving to this cell from another cell

Connected: flag to let know if this should be connected in the graph if false, no edges will be drawn to the node and it will not be possible to move to that cell

Connections



Example of a Orthogonal Square Graph



Example of typical Hexagonal Cube Coords Graph

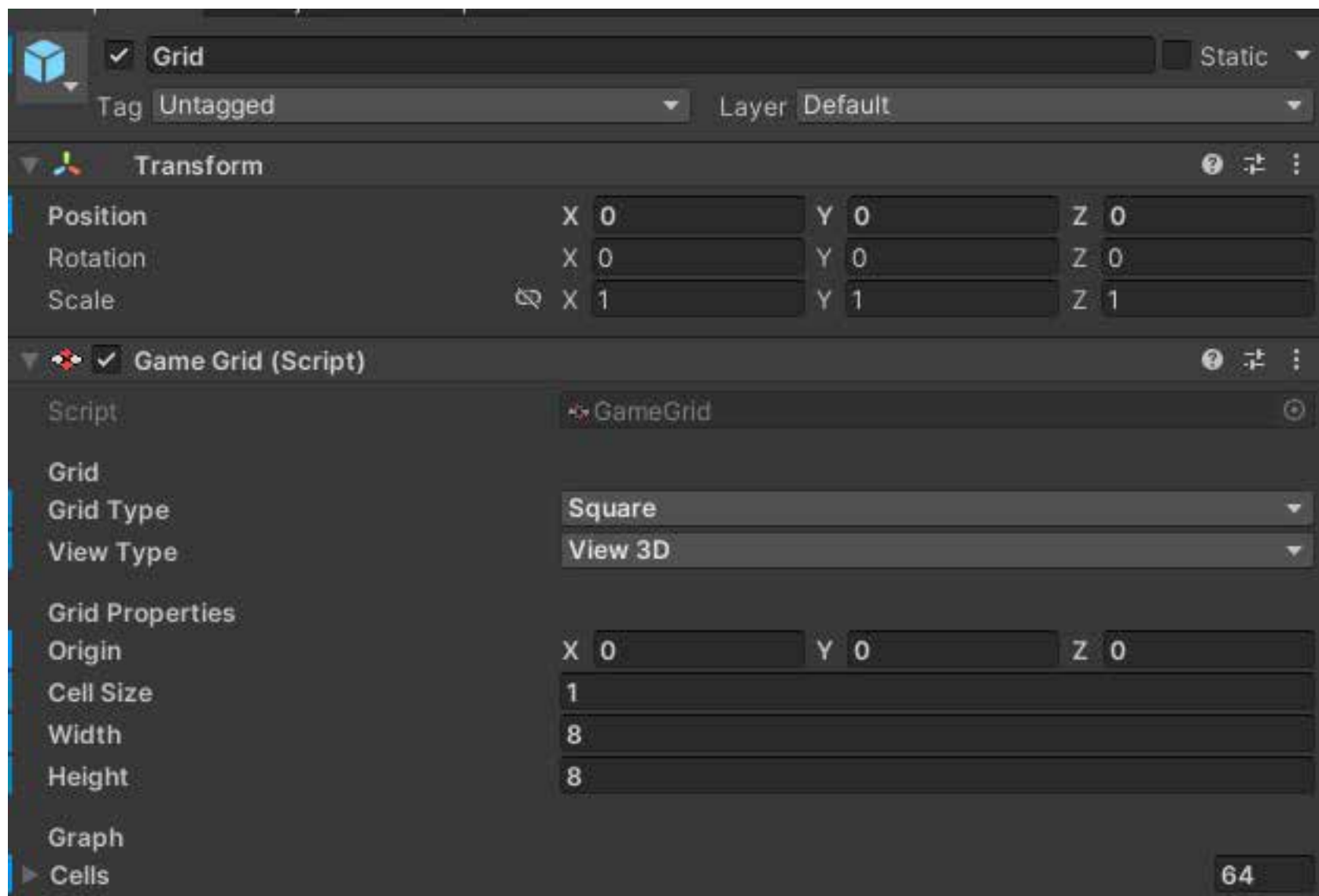
The Cell object can be interacted with via the ISelectable interface. If initialized with default settings, the game will try and find a way of detecting cells automatically, and it is up to you if you want to use a collider or not. It is recommended you use a collider for 3D cells. If no collider is found it will try and retrieve a cell based on a plane at the origin of the grid.



Game Grid Component

The Game Grid is responsible for storing the cells of the game, as well as building and maintaining a graph of cells and their connections. Most other classes will reference the Game Grid to be able to manipulate some of the cells' state, such as paths between cells. It also references a list of all the entities in the scene and updates the grid based on the entries in that list

A typical Game Grid



Grid Type: An enum telling us if the grid is a square grid or a hexagonal grid

View Type: An enum telling us whether the game is in 3D or 2D view

Origin: The position of the grid

Cell Size: The size of the cells in the grid. Measured as outer radius if it's a hexagonal grid

Width: Measured as amount of cells in a row of the grid

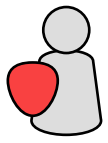
Height: Measured as amount of cells in a column of the grid

Entity Set: A data asset containing all the active entities in the scene

Cells: All the cells in the grid

Before the game is started you will have to build the Graph of the game grid, which you can do using the BuildGameGrid method. This is done by default in the Initializer.

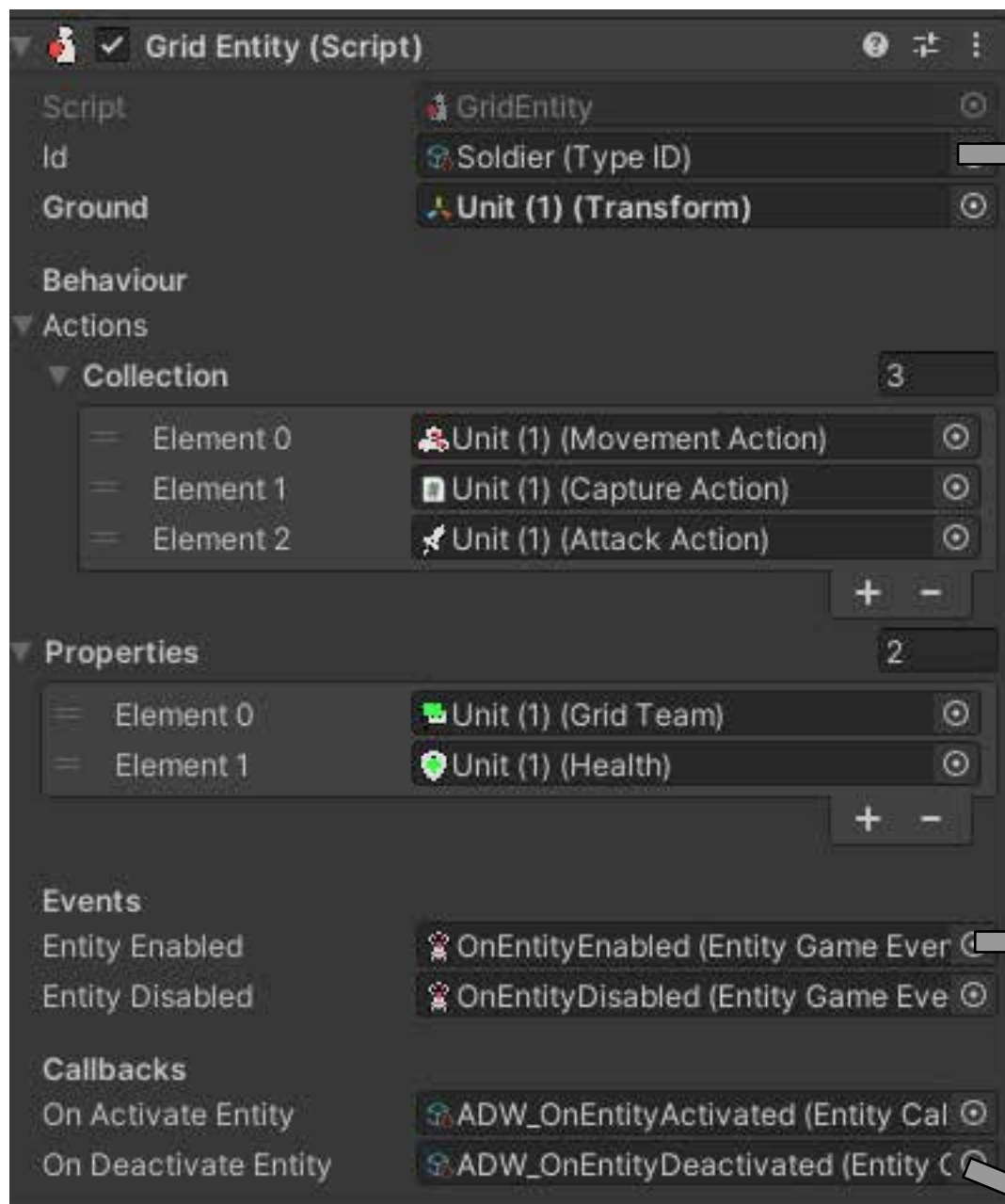
If you are using the default Initializer to start up your game it is important that GridType and ViewType is set correctly as it will try to figure out methods of selecting based on those values :)



Grid Entity Component

The Grid Entity represents a unit or character that the player can interact with. You can interact with it using the ISelectable interface to return action states from the actions list. The Grid Entity will not do much on it's own but you can populate the actions and properties lists to create new and different behaviour.

A typical Grid Entity



Just a empty Scriptable Object that works as an identifier.

In the samples these events are raised to let the Initializer (or your implementation) know when there is a new Entity in the scene. The Entity is passed as a paramter to the event, and the Initializer will provide it with dependencies.

Id: A TypedId data asset

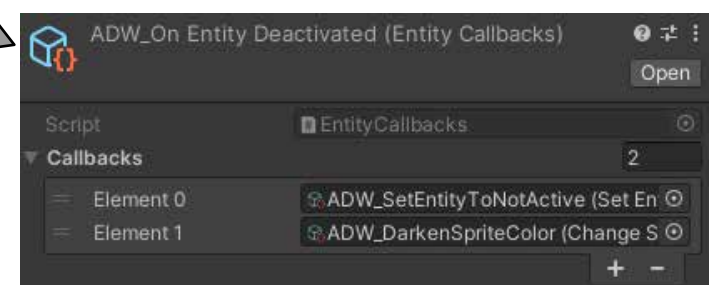
Ground: Transform where the entity is supposed to have it's 'feet'

Actions: The GridActions of the entity. Will be returned on order usign the Select() method

Properties: Dynamic properties and data that can be attached to the Entity. Typically used for additional functionality that is unique to each Entity and interacts with different actions - such as Health, Team etc.

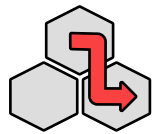
Events: Events raised during on the OnEnable/OnDisable callbacks.

Callbacks: List of callbacks that will be invoked when Activate/Deactivate is invoked on the entity



An EntityCallbacks object, which will invoke a list of EntityCallback. These are callbacks that you can reference in the inspector and create as assets. Thus making it easy to modify and extend what happens when a method is called.

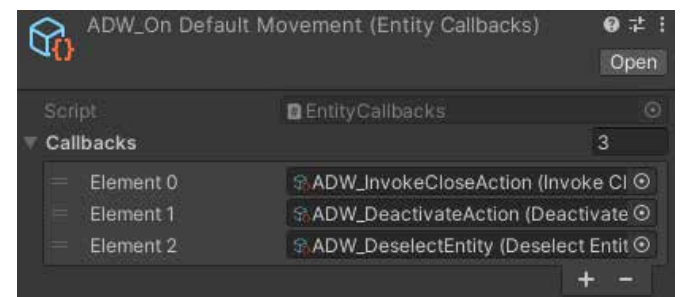
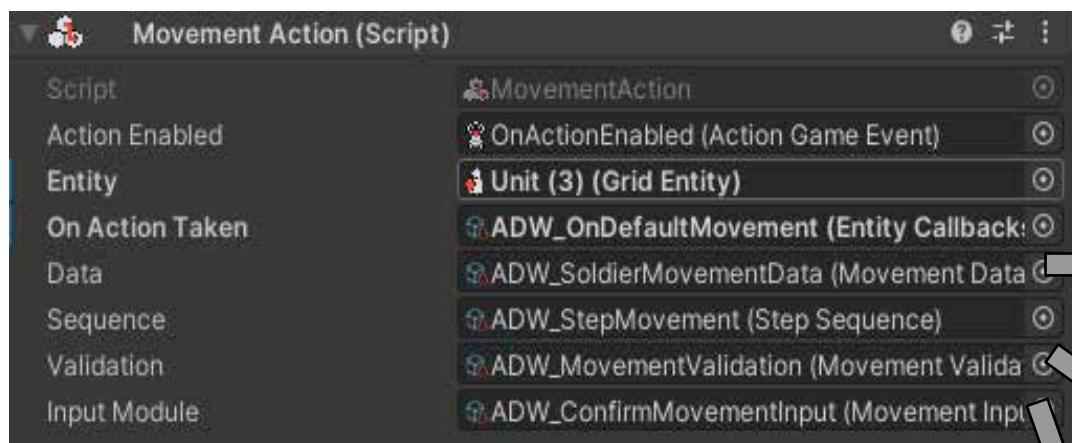
You can extend the EntityCallback class to create your own callback assets.



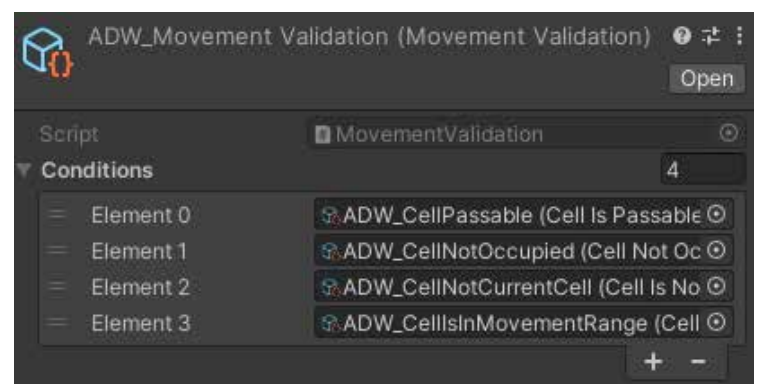
Grid Action Component

The Grid Action is the base class for any behaviour of an Entity. The MovementAction and AttackAction are in the base library and are examples of a Grid Action. It stores data and methods determining and executing moves on the grid. They can return a State class for handling user input, but can also be used directly, such as in the case of an AI opponent.

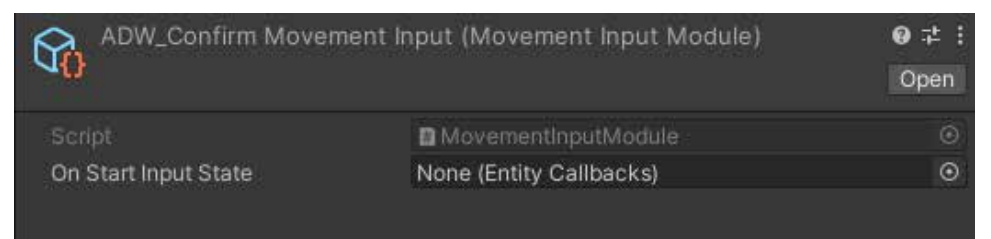
A typical Grid Action



A list of callbacks to be invoked when a move is taken.



A list of callbacks that all need to return true in order for a move to validate.



An asset that returns a state for confirming the action via user input. You can also add a list of callbacks to invoke when the input state gets started

Action Enabled: Event raised to signal that the action has become enabled in the scene.

Entity: The GridEntity, or owner of the action.

On Action Taken: Callbacks invoked on calling the HandleActionTaken method.

--- Movement Only ---

Data: Data asset required for movement action

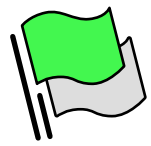
Sequence: Movemen sequence to be played when making a move on the grid

Validation: List of callbacks to validate a particular move

You can retrieve a State from the Grid Action using the Get() method - The default implementation for the MovementAction class will provide a ConfirmMovementState, that will handl visualization and input for a human player.

You can use an action by calling the StartAction() method, which should configure the action and grid for making a move.

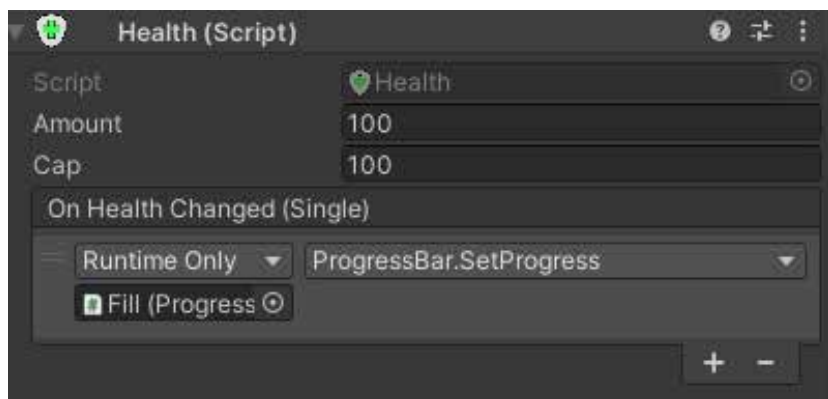
Have a look at some of the examples for inspiration on how you can use the GridAction class to create new behaviours. In the ADW_Sample project the Capture behaviour is implemented as a GridAction.



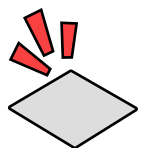
Grid Property Component

The Grid Property is just like any other MonoBehaviour and can be attached to game objects to contain dynamic data which will be manipulated during the game - such as Health, Team, etc. You can use the IPropertyHolder interface to search Grid Property by type, and the PropertyUtil helps out with logic between IPropertyHolder objects.

A typical Grid Property



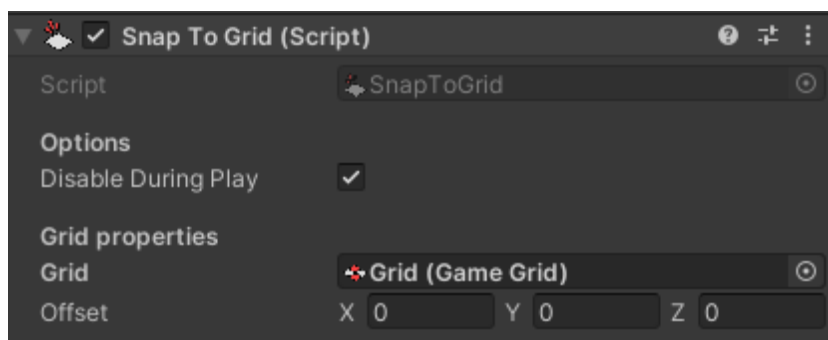
A Grid Property does not have any required information, and can be whatever you want it to be. It is created this way to enable quick lookup on an Entity, or other IPropertyHolder object. You could just as well get a regular component using GetComponent methods.



Snap To Grid Component

The Snap To Grid component allows you to place objects in the scene and snap them to the center of the closest cell.

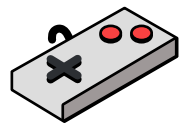
A typical Snap To Grid



Disable During Play: Flag to indicate whether to snap while in play mode. Recommended to be made as true.

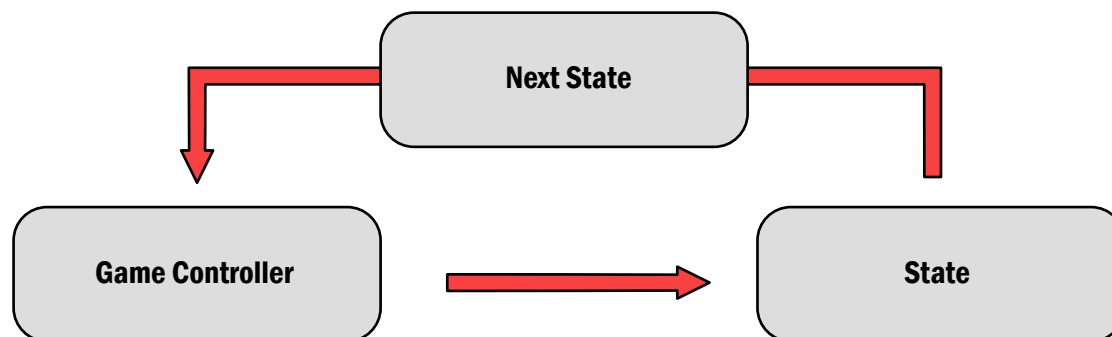
Grid: The game grid to snap to

Offset: Added to the transform.position of the Game Object



Game Controller Component

The Game Controller is responsible for running the core game loop by transitioning between game states using the so-called State Pattern.



The game loop will look something like this, where the Game Controller is responsible for running the current state, and the current state is responsible for providing the next state when it reaches a conclusion. Using the default implementation, returning null will result in running the IdleState which is a simple default class that listens for input from the player.

The State Class

The State class is an implementation of the State pattern and you can inherit this class to create new states and transitions. One example is the GridActions, that provide states using the ISelectable interface to the GameController when selected.

State provides three key methods for progressing, namely the Enter, Update and Exit methods. It also stores a reference to the next state. Enter will be called when you first run the state, then Update will run until Exit is called, where the next state will be returned and the cycle repeats.

To create your own state you can inherit the State class. Provide any necessary parameters in the constructor. and override the Enter, Update and Exit methods. Remember to also call Enter and Exit in the base class!

```
public EndingTurnState(int millisecondsDelay, TurnManager turnManager)
{
    _millisecondsDelay = millisecondsDelay;
    _turnManager = turnManager;
    _stopwatch = new Stopwatch();
}

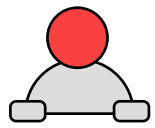
protected override void Enter()
{
    _turnManager.EndTurn();
    _stopwatch.Start();
    base.Enter();
}

protected override void Update()
{
    if(_stopwatch.Elapsed.Milliseconds < _millisecondsDelay)
        return;

    nextState = _turnManager.currentPlayer.PlayerStartTurnState;
    _turnManager.StartTurn();
    Exit();
}
```

Example of the simple EndTurnState class - a state used when ending the turn and passing it to the next player.

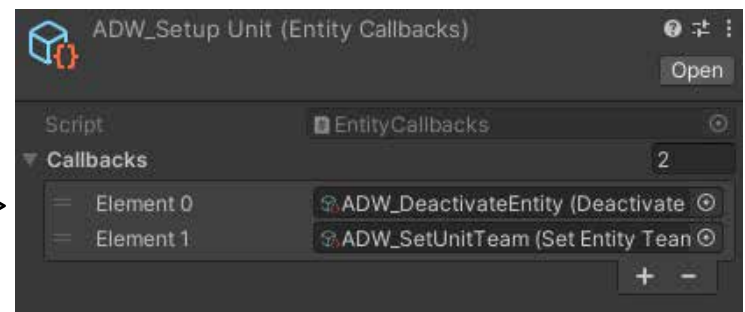
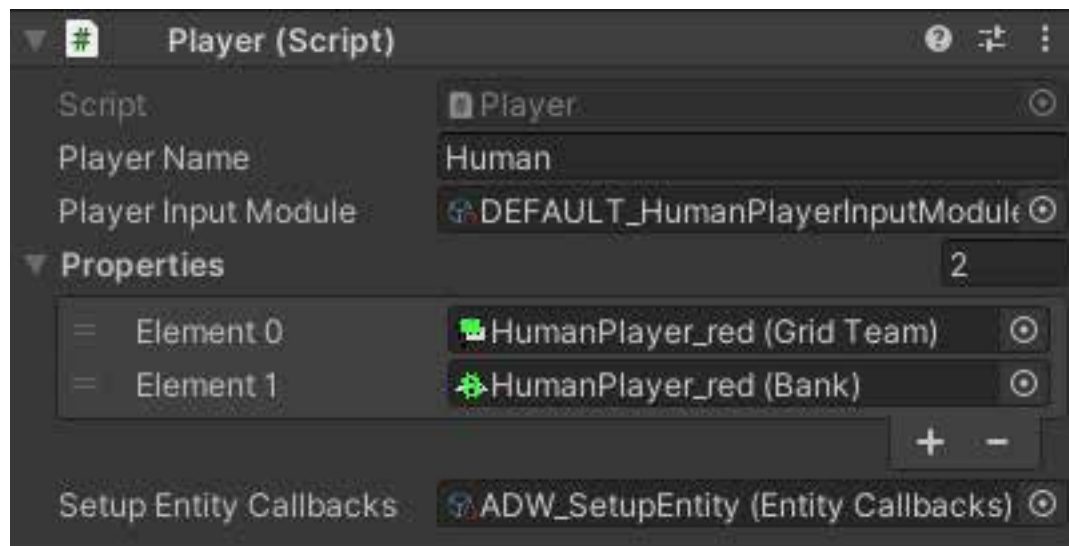
- The Enter method invokes EndTurn on the Turn Manager
- The Update method checks if enough time has passed, and if it has, it fetches the next state from the Turn Manager and invokes the StartTurn.
- This state will return 'nextState', which happens to be the start state of the player that will start the new turn.



Player Component

The Player Component is a simple class representing a player that controls the game during a turn. It implements the IProperty-Holder interface and can carry properties just like a Grid Entity. It's main responsibility is setting up entities using the SetupEntity method. It also needs to supply a starting state to the game when the player enters it's turn. You can add state using the PlayerInput-Module-class to determine if it's a human player, ai player, etc.

A typical Player



A list of callbacks that will be invoked when the SetupEntity method is called.

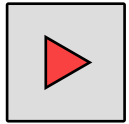
This implementation is from the ADW_sample where any time a unit is created it is provided with a team and deactivated until the next turn.

The PlayerInputModule class can be inherited to provide a starting state for a player.

```
No asset usages 4 usages 3 inheritors 1 Calle
public abstract class PlayerInputModule : ScriptableObject
{
    1 usage 3 overrides 1 Calle
    public abstract State Get(Player pLayer);
}
```

As an example: If you add this PlayerInputModule Asset to the player class, when it's the players turn, the game will run the DefaultAiState. You can create any type of player from the regular Player class using different input modules to control the player states.

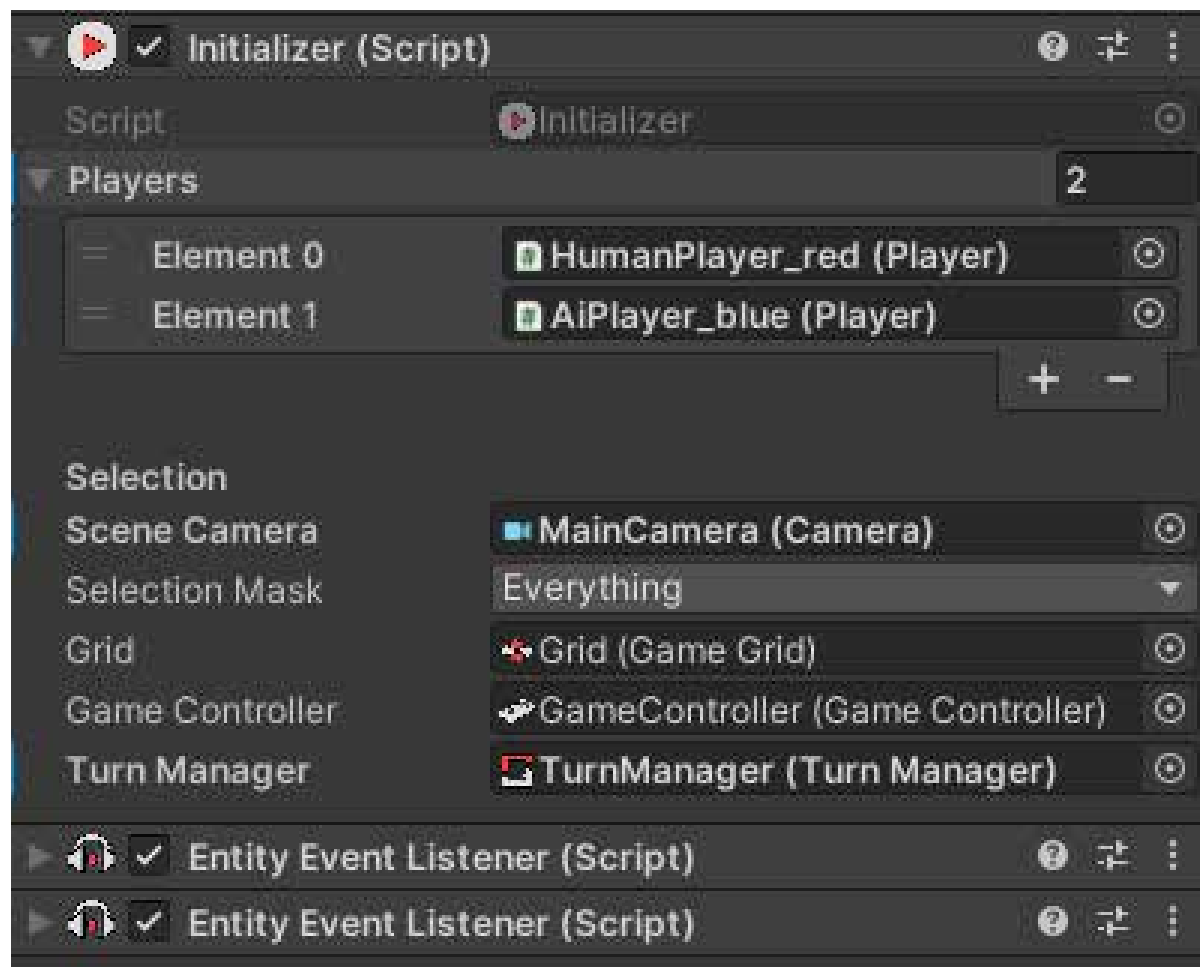
```
[CreateAssetMenu(menuName = "Gridr/Input Module/Player/Ai Player Input Module")]
1 asset usage 1 Calle
public class AiPlayerInputModule : PlayerInputModule
{
    0+1 usages 1 Calle
    public override State Get(Player player)
    {
        return new DefaultAiState(player);
    }
}
```

Initializer Component

The Initializer is responsible for providing dependencies and booting up the game in the scene. If you want to provide different ways of handling input, or listeners you can provide other implementations here. The Initializer will try and create the best ways of listening for cells and units based on the settings of the grid, this is mostly to enable to quickly move between either hex, square or 2D or 3D without needing new setups. Once/If you know what kind of project you are creating it is appropriate to provide the specific implementations needed instead.

A typical Initializer



→ Add all your players to the initializer to handle turns, etc. If not player is added you will be able to control all units without turns.

→ You can add any components or objects that need to be initialized as a public, or serialized field.

→ Using the event system you can have objects initialized on the spot. In the examples, the Initializer will run the Init methods on Entities in response to them raising the OnEntityEnabled event. This can come in handy if you want to have multile grid styles whilst maintaining the same GameObjects. Have a look in the class to see what is provided to Entities and Actions

The initializer handles initialization of all Input Modules in the project. If you don't want to initialize sample assets, you can delete them or put them in a separate folder.



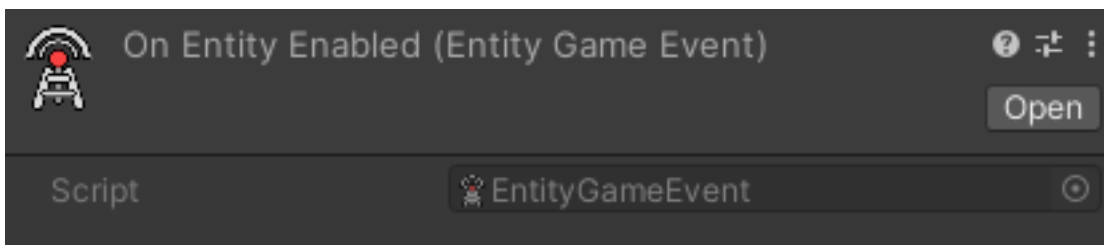
Event System

The event system is implemented using Scriptable Objects, which allows for creating events as a data asset and having them referenced as a field in the inspector.

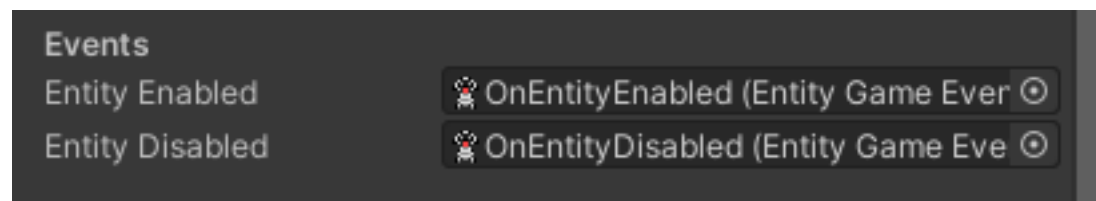
A Typical Event

```
[CreateAssetMenu(menuName = "Events/Entity Event")]  
  
public class EntityGameEvent : GameEvent<GridEntity> { }
```

To create an event you can inherit the `GameEvent<T>` class and specify what kind of data you want to pass as a parameter.



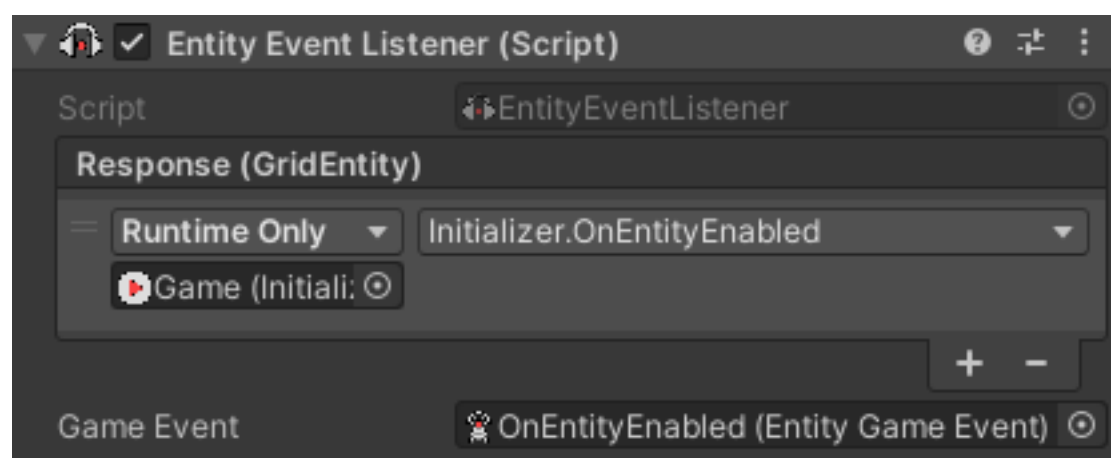
Once your class is created, you can create a event asset using the create menu.



To raise the event from a `GameObject` you can add a field of the desired type and pass in the new event asset through the inspector. The event can then be raised using the `event.Raise()` method.

```
public class EntityEventListener : GameEventListener<GridEntity>  
{  
    public EntityGameEvent gameEvent;   
    public void OnEnable() => gameEvent.AddListener(this);  
    public void OnDisable() => gameEvent.RemoveListener(this);  
}
```

To listen and respond to events you will need a corresponding `EventListener`.



To Respond to the event you can attached your `EventListener`, specify the event you want to respond to and add responses.



Entity Callbacks

The Entity Callback is an abstract class that you can use to manipulate in a variety of ways and it takes a bunch of optional parameters. When you inherit the EntityCallback class, since it's a Scriptable Object, make sure to include the CreateAssetMenu attribute.

* ActionCondition / Validation follow the same pattern

A Typical Callback

```
[CreateAssetMenu(menuName = "Unit Callbacks/Change Sprite Color Callback")]
2 asset usages 1 Calle
public class ChangeSpriteColor : EntityCallback
{
    public Color color; 1 Serializable

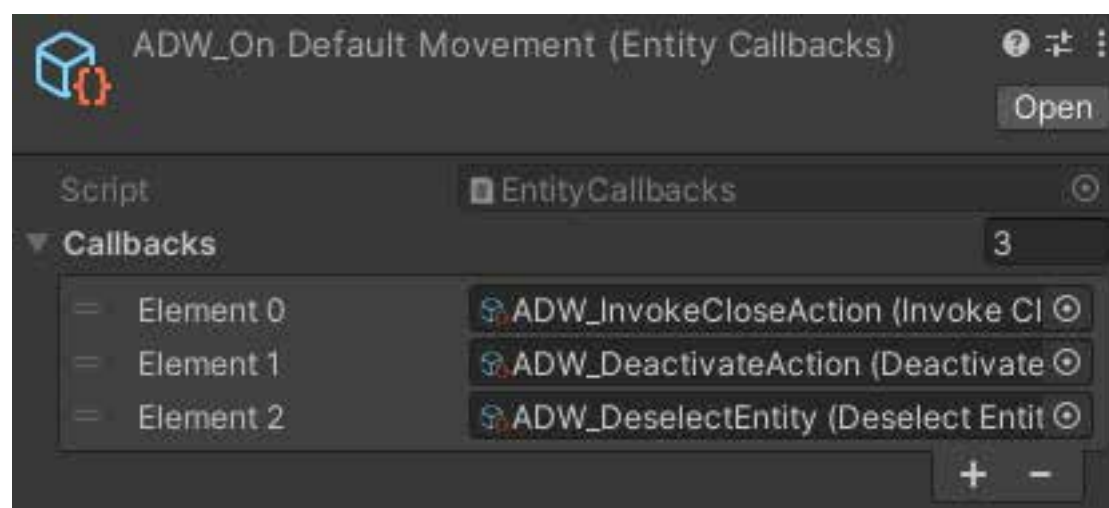
    1 Frequently called 0+1 usages 1 Calle
    public override void Invoke(GridEntity entity, Player player = null, GridAction gridAction = null, GridProperty property = null, Cell cell = null)
    {
        if(!entity)
            return;

        var spriteRenderer = entity.GetComponent<SpriteRenderer>();
        if (spriteRenderer)
            spriteRenderer.color = color;
    }
}
```

The callbacks are good to use when you want to do specific operations or logic on an element and want an overview in the inspector. This particular callback sets the color of the GameObject's spriterenderer.

How To Invoke?

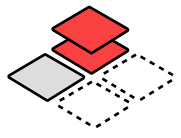
You can either create a direct reference to the object you wish to invoke in a script, but it is more useful to use them when you want to execute callbacks in sequence. To do that you simply need to reference a EntityCallbacks object (notice the s), which is also an asset file and run the InvokeAll method.



Each collection of callbacks will have to be created using the create menu, and then you can pass your callback assets in to the list in the order you want them to be invoked. In this example, the MovementAction will, once a move is completed:

- Invoke it's Close method
- Deactivate the action
- Deselect the Entity

This way you can drag and drop behaviour on to the action, even at runtime.



Grid Building Tool

The Gridr Framework comes with a grid building tool that you can find under the tools tab. It will allow you to more quickly build and debug your grids.

Controls:

- Press the scene-view preview buttons to select brush and drawing mode
- Press and hold Shift to Draw / Delete
- Fill will fill the empty part of the grid with the currently selected brush
- Clear will remove all the cells from the grid

The screenshot shows the 'Square Grid Builder' window with two tabs: 'Creativity' and 'Comfort'. The 'Creativity' tab is active. The window is divided into several sections:

- Top Section:** Contains 'Grid Builder' and 'Grid Builder' buttons. The 'Grid Builder' button is highlighted. The 'Grid Builder' button is annotated with an arrow pointing to it, stating: 'Sets the View Type, wheter you want to work on a 2D or 3D grid.'
- Info Section:** Displays grid statistics: 'Cells: 294', 'Width: 21', 'Height: 14', 'Cell Size: 1'. It also includes 'Load Grid' and 'Set Grid Positions' buttons.
- Origin Section:** Shows 'Origin' coordinates: 'X: -2', 'Y: -2', 'Z: 0'. It also includes 'Cell Size', 'Grid Width', and 'Grid Height' fields.
- Brush Section:** Displays the selected brush: 'ADW_Ground (Brushes)'. It is annotated with an arrow pointing to it, stating: 'The Brushes Data asset ist a collection of pre-fabs/cells you want to put on the board.'
- Game Grid Section:** Contains 'Game Grid' and 'Grid (Game Grid)' buttons. The 'Grid (Game Grid)' button is highlighted. It is annotated with an arrow pointing to it, stating: 'The Game Grid in the scene.'
- Build Graph Section:** Includes 'Build Graph' and 'Build Immediate' buttons. The 'Build Immediate' button is checked.
- Bottom Section:** Displays 'Info' statistics: 'Cells: 294' and 'Connections: 564'.

Additional annotations include:

- An arrow pointing to the 'Auto Size' checkbox, stating: 'If true will auto size the grid cell size based on your brush'.
- An arrow pointing to the 'Grid Width' field, stating: 'Sets the properties of the grid. Some of these values will try to resolve automatically when you open the window'.

You can use the Comfort tab to adjust settings and optional debug-settings. If there is something not working with your grid, it can be handy to make sure the GridPositions indices look good, and that the graph edges are connected as you expect them to!

*** IMPORTANT: To be able to get a 2D preview of a sprite, you will have to enable the 'Read/Write' flag on your sprite texture ****

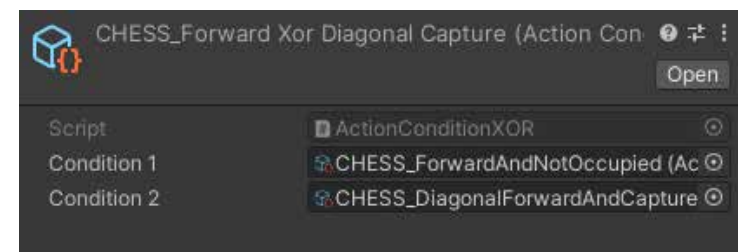
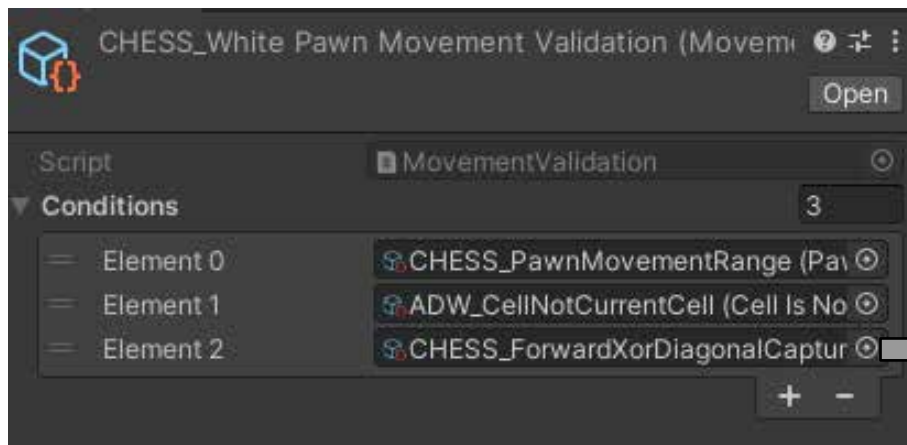
*** IMPORTANT: Unity version 2021.3.9 & 2021.3.10 Have a bug in which the program crashes on Undo PLEASE USE 2021.3.6 or LOWER to make use of the Grid Tool****



Action Validation

The ActionValidation helps you to validate actions. It will inspect a cell and the current Action, and return whether the Action can be performed on the cell. Typically it will consists of a list of ActionCondition that will be run on each cell in the grid when the actions is started. The ActionConditions are also created as assets and can be replaced and configured as you wish.

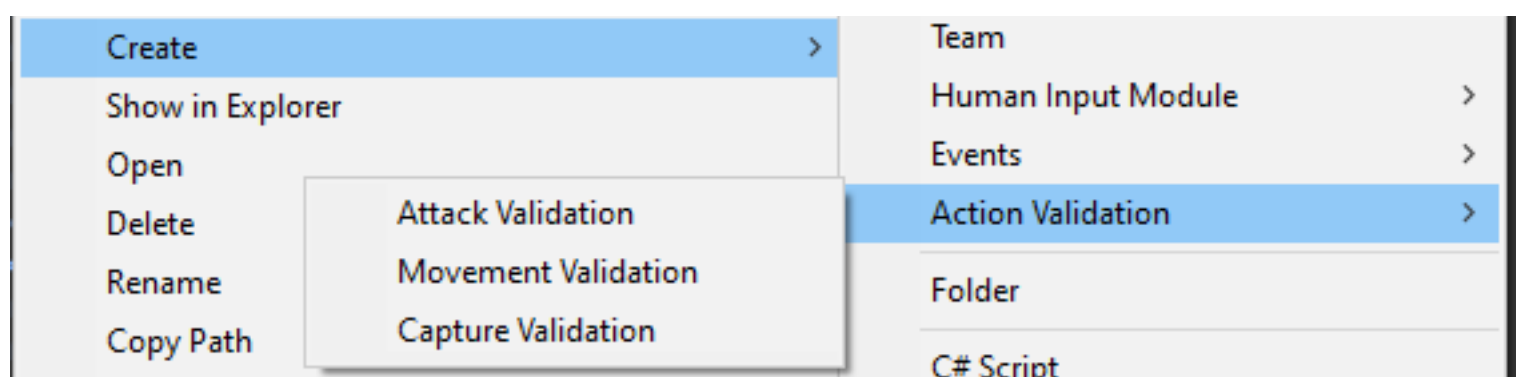
A typical ActionValidation



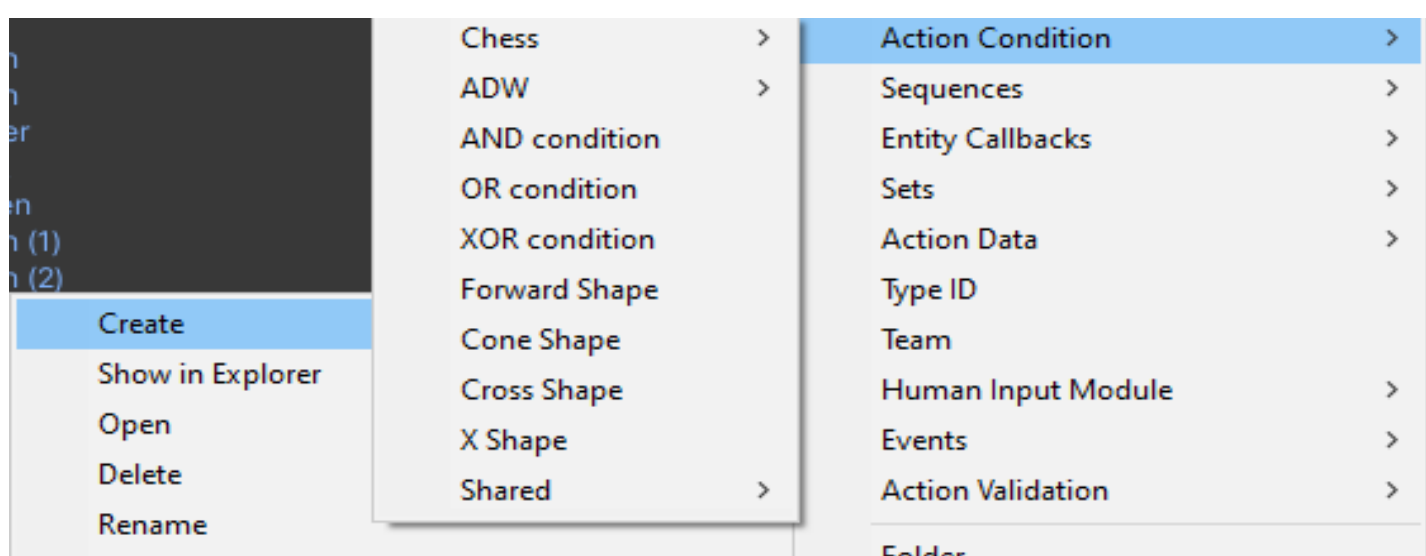
The conditions can consist of other lists of conditions. This particular condition only returns true given the XOR of the two conditions

All ActionConditions need to return true for the cell to be a valid move and you can create complex validation by creating a custom ActionCondition or build a tree of conditions using the AND, OR, XOR conditions. To create your own ActionCondition, just inherit the base class and provide a method for validating the cell.

To create a ActionValidation asset you can right click in your project and go to Create/Action Validation.



To create a ActionCondition asset: right click in project, Create/Action Condition.



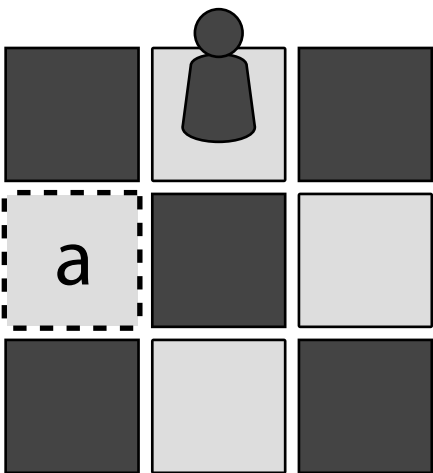


Action Validation

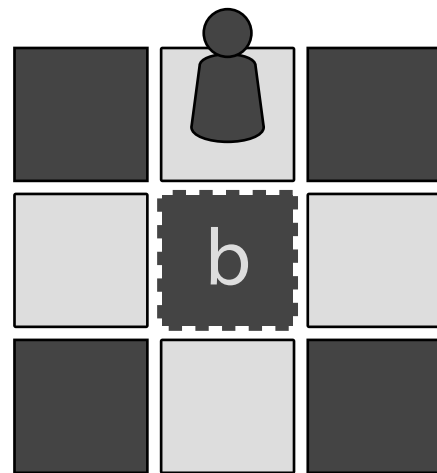
Example

Given the black pawn is a regular chess pawn that can move forward, or capture diagonally if the diagonal square is occupied by a white piece. First the black piece is selected, and the MovementAction will validate each cell. Illustrated below is an example of the validation validating cells a, b, and c.

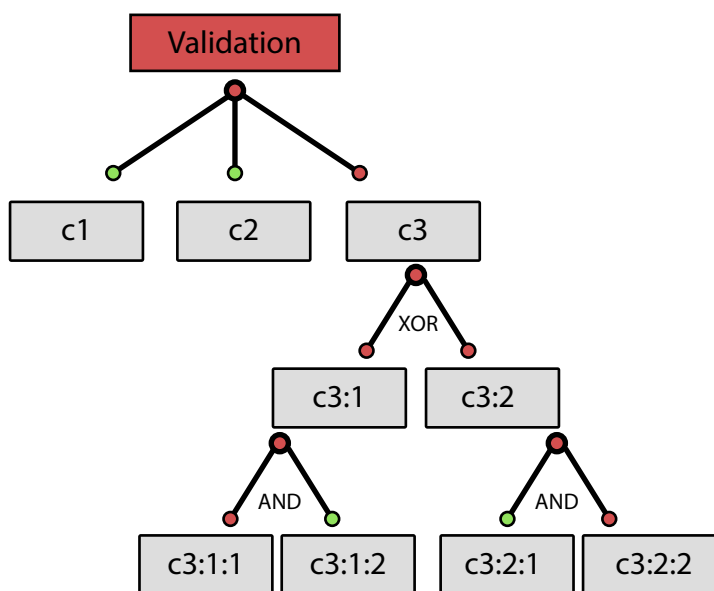
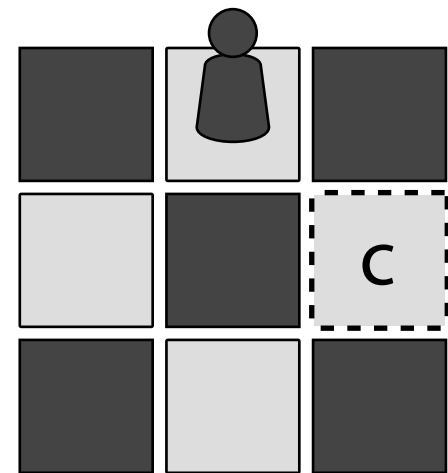
V1



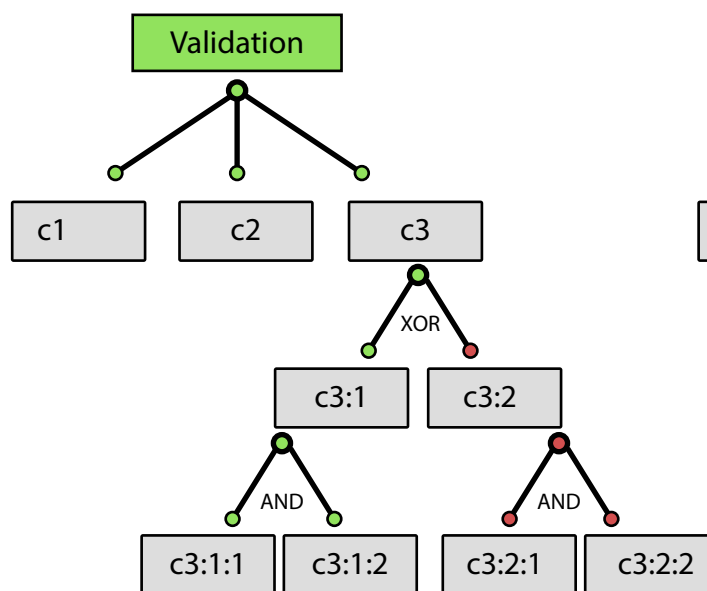
V2



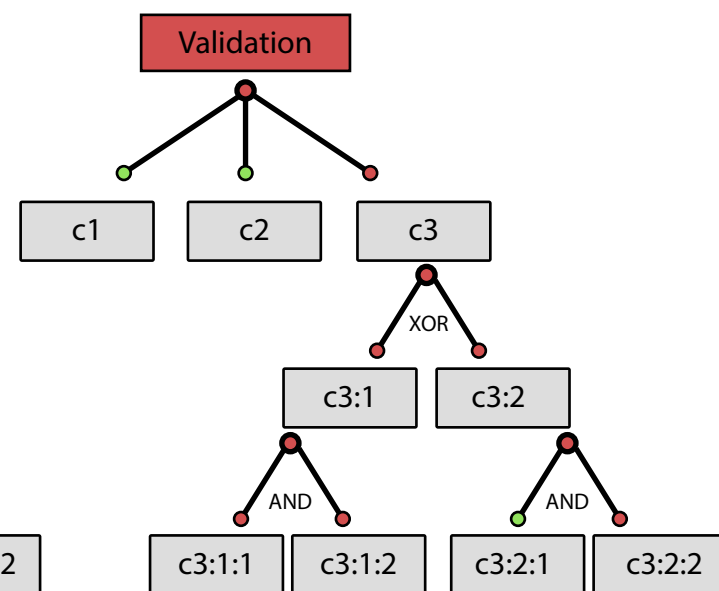
V3



false. Can not move here.



true. Can move here



false. Can not move here

Layer 1

c1: Cell is in movement range

c2: Cell is not current cell

c3: Cell is forward XOR diagonal capture

Layer 2

c3:1: Cell is forward AND cell is not occupied

c3:2: Cell is diagonal forward AND can capture on cell

Layer 3

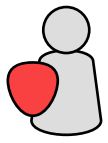
c3:1:1 Cell is forward

c3:1:2 Cell is not occupied by any pawn

c3:2:1 Cell is diagonal forward

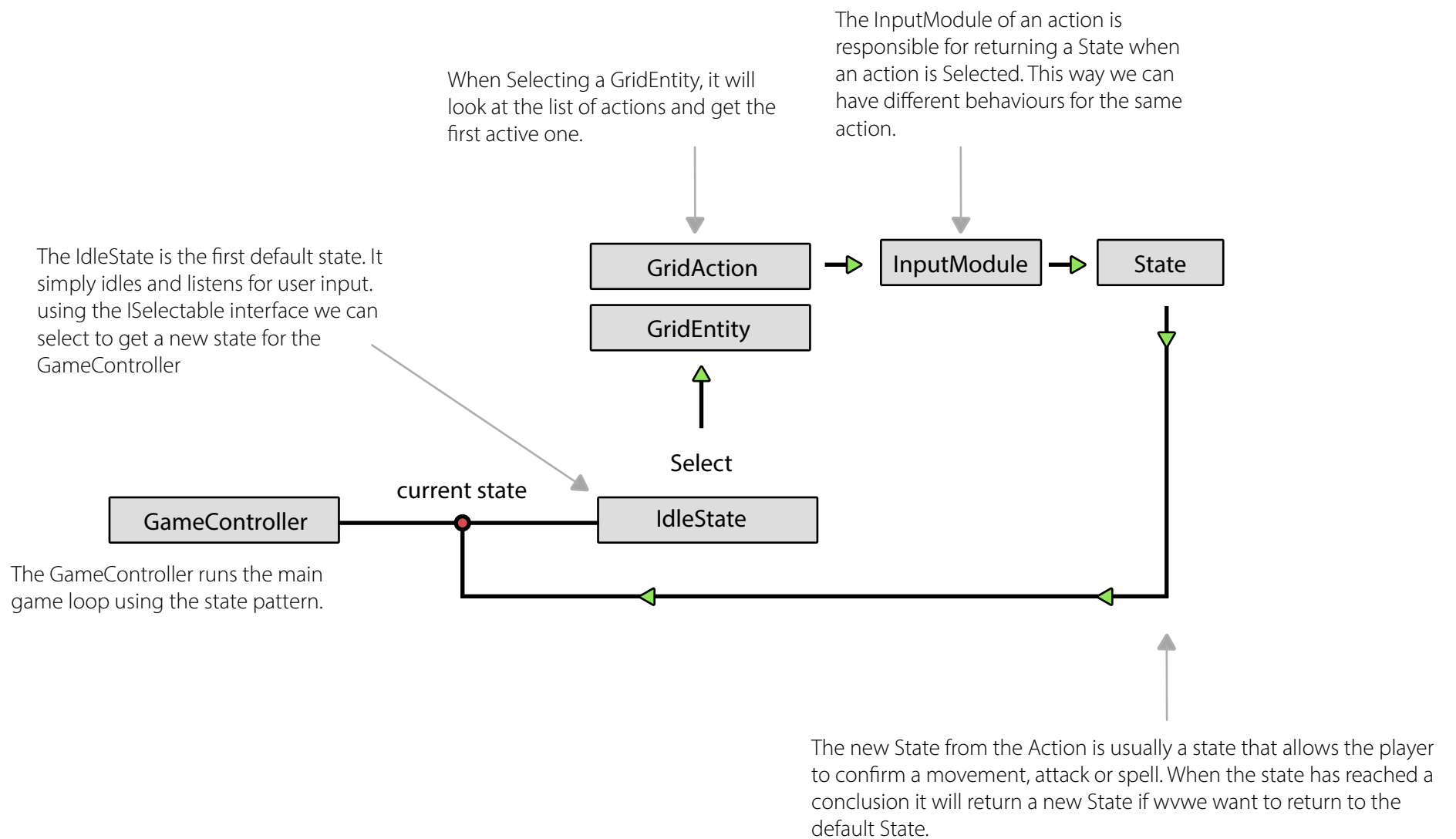
c3:2:2 Cell is occupied by opponent pawn

You can layer validation assets piece by piece like in this example. Or if you are more comfortable with just code you can add all the validation in to a single asset. But doing so you will have to change the code to change the validation, thus removing some flexibility.



Grid Entity Component

The GridEntity provides interaction by returning states to the GameController. We can have one or more GridEntity on a single cell, but doing so we will need to somehow sort the results from the `IListener<ISelectable>`. When selecting the GridEntity from an IdleState, it will return the states provided by the InputModule for each GridAction on the GridEntity. Have a look at the chart below to get an overview of how the GameController and a GridEntity with GridActions are put together.





Thank You

<https://www.generalgames.org>

<https://bsky.app/profile/generalgames.bsky.social>

<https://twitter.com/generalgames9>

<https://generalgames.itch.io/>

<https://discord.gg/PyQP7MC>